

Group A report

Milestone 5: Dynamics & Animation

This milestone extends the procedural terrain generator developed in previous milestones by introducing **dynamic behavior, stochastic effects, and performance evaluation**. The previously generated terrain was static; therefore, this phase focused on transforming it into an interactive environment with animation, particle effects, and efficiency analysis.

The main objectives were:

- To introduce temporal behavior through terrain animation.
- To implement a stochastic particle system for environmental realism.
- To evaluate animation efficiency using performance metrics.
- To analyze system stability and visual realism.

Animation System Implementation

1. Camera Rotation Animation

A real-time animation system was implemented using the Matplotlib FuncAnimation framework. The camera position changes gradually across frames, creating a rotating view around the procedural terrain.

Implementation concept:

At every frame:

1. The previous frame is cleared.
2. The terrain is redrawn.
3. The camera angle is updated.
4. The next frame is generated.

Result:

The terrain can be viewed from different angles, allowing interactive exploration of the generated landscape.

2. Procedural Water Animation

In addition to camera movement, procedural deformation was implemented for areas below the defined sea level.

The water movement is generated using sinusoidal functions. The wave displacement is calculated using:

$$[\\ W(x, z, t) = A \sin(0.5x + t) \cos(0.5z + t) \\]$$

Where ($A=0.15$), (x, z) are terrain coordinates and (t) is animation time.

The deformation is applied only to points below sea level:

water = $Z < \text{self.sea_level}$

$Z[\text{water}] += \text{wave}[\text{water}]$

This keeps the mountains and dry terrain stable while creating continuous movement in the lake. The purpose was to demonstrate procedural deformation and temporal behaviour within the terrain system.

Purpose

The procedural water animation was introduced to:

- Simulate environmental movement.
- Demonstrate temporal procedural behaviour.
- Produce smooth rather than discontinuous motion.
- Maintain consistency with the generated terrain.
- Demonstrate deformation-based animation.

3. Motion Consistency Analysis

The animation uses continuous mathematical functions rather than random frame-by-frame movement.

The camera angle changes gradually according to the interpolation equation, preventing sudden changes in viewing direction.

The water deformation is also based on continuous sine and cosine functions. Therefore, the displacement changes smoothly over time.

This provides temporal consistency because consecutive frames are mathematically related rather than independently generated.

Motion Characteristics

Characteristic	Result
Camera movement	Smooth
Sudden camera jumps	None during normal movement
Camera direction	Controlled
Water movement	Continuous procedural deformation
Terrain geometry	Stable
Animation loop	Continuous

4. Animation Performance

During development, it was observed that redrawing the entire terrain and recalculating lighting on every frame produced slower animation.

The original approach performed the following operations repeatedly:

1. Clear the 3D plot.
2. Recalculate terrain deformation.
3. Recalculate lighting.
4. Recalculate surface colours.
5. Redraw the entire terrain.
6. Update the camera.

This created unnecessary computational overhead.

The animation system was therefore improved by rendering the terrain once for camera-only animation and changing only the camera viewing angle between frames.

The optimized camera animation performs substantially less computation because the terrain does not need to be reconstructed for every camera frame.

Performance Observation

Animation approach	Observation
Rebuilding terrain every frame	Slower
Recalculating lighting every frame	Computationally expensive
Rendering terrain once	Faster
Updating camera angle only	Smoothest camera movement

5. Stability Evaluation

The animation system was tested during continuous execution.

The following criteria were evaluated:

Criterion	Result
Program crashes	No
Terrain rendering errors	No
Camera movement	Stable
Sudden camera movement	No during normal interpolation
Terrain geometry	Stable
Water deformation	Successful
Animation looping	Successful

6. Visual Realism Evaluation

The procedural terrain already contains multiple levels of height variation, including large mountain formations, medium-scale hills, rolling terrain and fine details.

Milestone 5 adds temporal behaviour to this static environment.

Terrain realism

- Large-scale mountain formations.
- Medium-scale hills.
- Rolling terrain.
- Fine surface variation.
- Central lake depression.

Animation realism

- Smooth camera movement.
- Controlled viewing direction.
- Continuous procedural water deformation.
- No sudden terrain displacement.

Environmental behaviour

The water deformation introduces a simple procedural environmental effect. The use of sinusoidal functions provides predictable and continuous movement while remaining computationally simple.

8. Limitations

The current animation system uses Matplotlib, which is primarily a scientific visualization library rather than a real-time graphics engine. Consequently, the animation is not expected to achieve the frame rates of dedicated real-time graphics frameworks.

Another limitation is that the camera animation and terrain deformation have different computational costs. Camera-only movement is significantly cheaper because the terrain can remain unchanged, while deforming the terrain requires additional surface calculations.

The current system also does not implement a particle-based environmental system. Particle effects were considered as a possible extension but were not included in the final implementation.

9. Conclusion

Milestone 5 successfully introduced temporal behaviour into the procedural terrain generation system through procedural camera animation and controlled water deformation.

